

Pseudo-temporal ordering and RNA velocity – Practical session

Zhisong He, PhD
Senior Scientist and Lecturer
Treutlein lab, D-BSSE, ETH Zurich
Basel, Switzerland
20.05.2026



Outlines

- **Introduction**
 - Setup your environment (Python)
 - The example data set
- **Analysis**
 - Brief overview on the data
 - Diffusion pseudotime
 - Coarse-grained trajectory analysis with PAGA
 - RNA velocity analysis with scVelo
 - Fate probability estimation with CellRank 2

Set up your work environment

Packages in need:

- Python
 - scanpy
 - scvelo
 - cellrank
 - (pyurd)

Set up local conda environment:

After installing conda (miniconda/miniforge/anaconda)...

```
> conda create -n env_embo_pseudotime python=3.11
> conda activate env_embo_pseudotime
> git clone https://github.com/GIMM-BioCode/2026_Pipette2Code.git
> cd 2026_Pipette2Code/02_Trajectories_ZHe
> pip install -r requirements.txt
```

Recommendation for Windows users

Install a Linux environment with WSL2

Use Google Colab

Register a Google account if you don't have one. Afterwards, login and visit the Colab website: <https://colab.research.google.com/>. Create a new notebook and open. In the notebook, input and execute the follows:

```
!git clone https://github.com/GIMM-BioCode/2026_Pipette2Code.git
!pip install -q -r 2026_Pipette2Code/02_Trajectories_ZHe/requirements-colab.txt git+https://github.com/zhisonghe/pyurd.git
```

More information can be seen here:

https://github.com/GIMM-BioCode/2026_Pipette2Code/tree/main/02_Trajectories_ZHe/README.md

Online vignette for the analysis

https://github.com/GIMM-BioCode/2026_Pipette2Code/tree/main/02_Trajectories_ZHe/

Files

main

Go to file

01_Introduction_TGomes

02_Trajectories_ZHe

03_Integration_SKoplev

04_ImmuneReceptors_AMaceiras

.gitignore

.python-version

README.md

Tutorial.ipynb

Tutorial_cleared.ipynb

pyproject.toml

requirements-colab.txt

requirements.txt

uv.lock

2026_Pipette2Code / 02_Trajectories_ZHe

zhisonghe Rename notebook and update README 7etcc18 · 23 minutes ago History

Name	Last commit message	Last commit date
..		
.gitignore	Add pyURD trajectory analysis section and environment setup	1 hour ago
.python-version	Add pyURD trajectory analysis section and environment setup	1 hour ago
README.md	Rename notebook and update README	23 minutes ago
Tutorial.ipynb	Rename notebook and update README	23 minutes ago
Tutorial_cleared.ipynb	Rename notebook and update README	23 minutes ago
requirements-colab.txt	Add pyURD trajectory analysis section and environment setup	1 hour ago
requirements.txt	Add requirements-colab.txt for Google Colab compatibility	1 hour ago
uv.lock	Add pyURD trajectory analysis section and environment setup	1 hour ago

README.md

Trajectory and velocity analysis

This session is handled by [Zhisong He](#)

Both codes and outputs

2026_Pipette2Code / 02_Trajectories_ZHe / Tutorial.ipynb

8.5 MB

```
sc.pp.neighbors(adata_full)
sc.tl.umap(adata_full)
sc.tl.leiden(adata_full, resolution=1, flavor='igraph', n_iterations = 2)

/Users/hezhi/Tools/2026_Pipette2Code/02_Trajectories_ZHe/.venv/lib/python3.11/site-packages/tqdm/autoraise.py:21: TqdmWarning:
IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm

In [4]: with rc_context({'figure.figsize': (3, 3)}):
sc.pl.umap(adata_full, color=['SOX2', 'DCX', 'FOXG1', 'EMX1', 'DLX2', 'MKI67'],
ncols=3, frameon=False, show=True)
sc.pl.umap(adata_full, color=['leiden', 'region', 'celltype'],
ncols=3, frameon=False, show=True, wspace = 0.7)
```

SOX2

DCX

FOXG1

Code only

2026_Pipette2Code / 02_Trajectories_ZHe / Tutorial_cleared.ipynb

Preview Code Blame 1175 Lines (1175 loc) · 38.6 KB

Rerun the preprocessing of the data

This section is to rerun data preprocessing, including normalization, HVG identification, PCA, clustering and cell type annotation, in case you want to get more familiar with how the data was processed and analyzed before doing the trajectory analysis.

Note

In AnnData, it is required that `.X` and all matrices in `.layers` should share exactly the same dimensionalities. Therefore, in the `DS1.h5ad` file, only the shared features of the total counts, spliced counts and unspliced counts matrices are kept. Meanwhile, the standard analytical pipeline takes the total counts matrix to start with. Therefore, we will load the other file, `DS1_raw.h5ad`. That file includes only the raw count matrix, plus the same `.obs` data frame as the `DS1.h5ad`.

```
In [ ]: adata_full = sc.read_h5ad('data/DS1_raw.h5ad')
adata_full.layers['counts'] = adata_full.X.copy()
sc.pp.normalize_total(adata_full, target_sum=1e4)
sc.pp.log1p(adata_full)
sc.pp.highly_variable_genes(adata_full, n_top_genes=5000, flavor='seurat_v3', layer='counts')
sc.pp.pca(adata_full, n_comps=30, mask_var='highly_variable')
sc.pp.neighbors(adata_full)
sc.tl.umap(adata_full)
sc.tl.leiden(adata_full, resolution=1, flavor='igraph', n_iterations = 2)

In [ ]: with rc_context({'figure.figsize': (3, 3)}):
sc.pl.umap(adata_full, color=['SOX2', 'DCX', 'FOXG1', 'EMX1', 'DLX2', 'MKI67'],
ncols=3, frameon=False, show=True)
sc.pl.umap(adata_full, color=['leiden', 'region', 'celltype'],
ncols=3, frameon=False, show=True, wspace = 0.7)
```

Note

Here are brief information of the genes being plot above:

Alternative vignette for the analysis

R: https://github.com/quadbio/scRNAseq_analysis_vignette

The screenshot shows the GitHub repository page for `scRNAseq_analysis_vignette`. The repository is public and has 82 forks and 262 stars. The main branch is `master`. The repository contains a `data` folder, an `images` folder, and files `README.md`, `Tutorial.md`, and `Tutorial.pdf`. The `README` file is selected, showing the title "Tutorial for scRNA-seq data analysis beginners using R". The text in the `README` states that the tutorial includes three parts: 1. Basic analysis using `Seurat` in R; 2. Data integration and batch effect correction; 3. Cell type annotation label transfer. It also mentions that the example data is from the paper "Organoid single-cell genomic atlas uncovers human-specific features of brain development".

Python: https://github.com/quadbio/scrnaseq_analysis_python_vignette/

The screenshot shows the GitHub repository page for `scrnaseq_analysis_python_vignette`. The repository is public and has 2 forks and 7 stars. The main branch is `main`. The repository contains a `data` folder, an `ext` folder, an `img` folder, and files `README.md`, `Tutorial.html`, `Tutorial.ipynb`, and `Tutorial.md`. The `README` file is selected, showing the title "Tutorial for scRNA-seq data analysis beginners in Python". The text in the `README` states that the tutorial includes three parts: 1. Basic analysis using `Scanpy` in Python; 2. Basic pseudotime, trajectory and fate probability analysis; 3. Data integration or batch effect correction. It also mentions that the example data is from the paper "Organoid single-cell genomic atlas uncovers human-specific features of brain development".

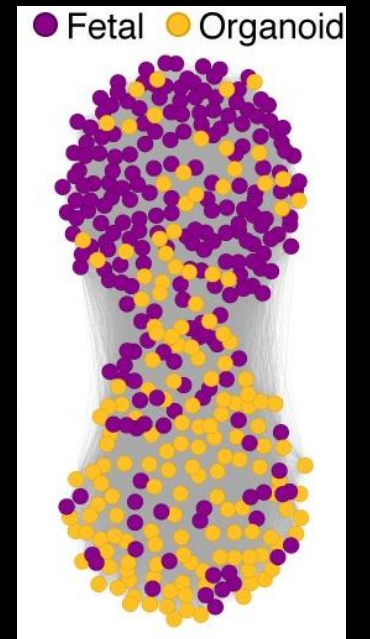
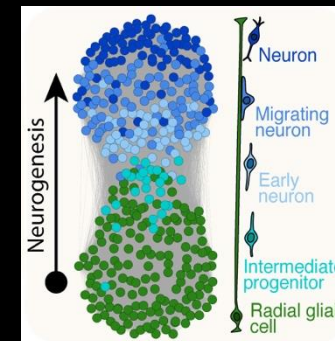
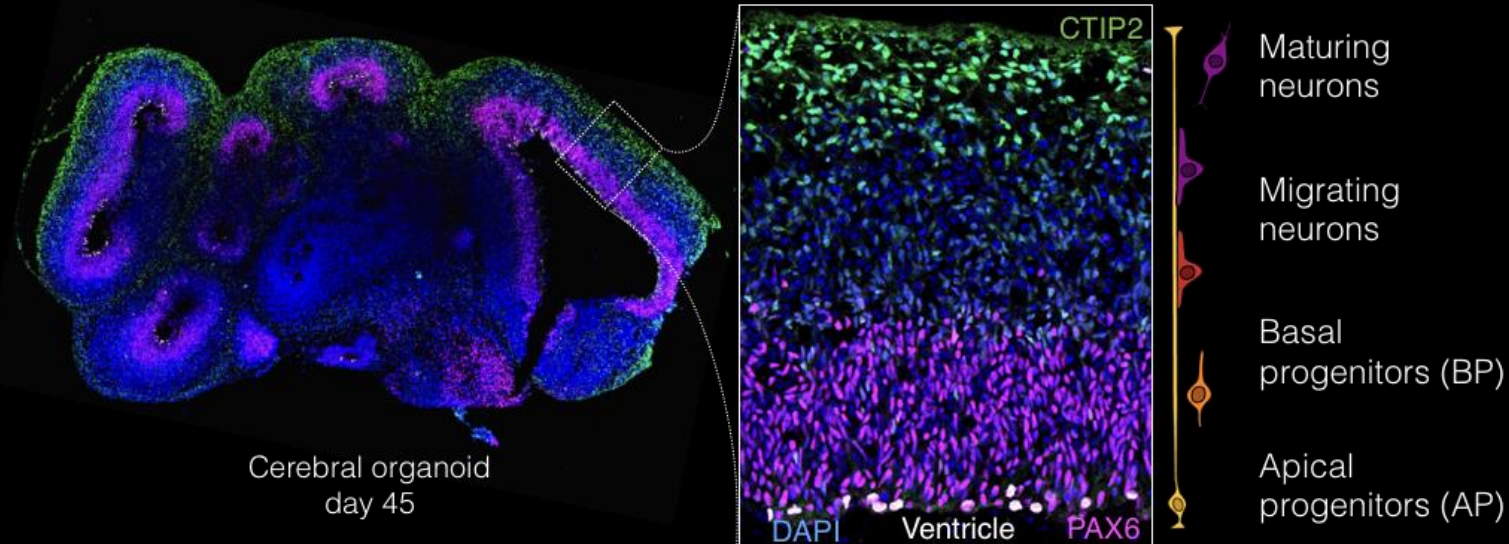
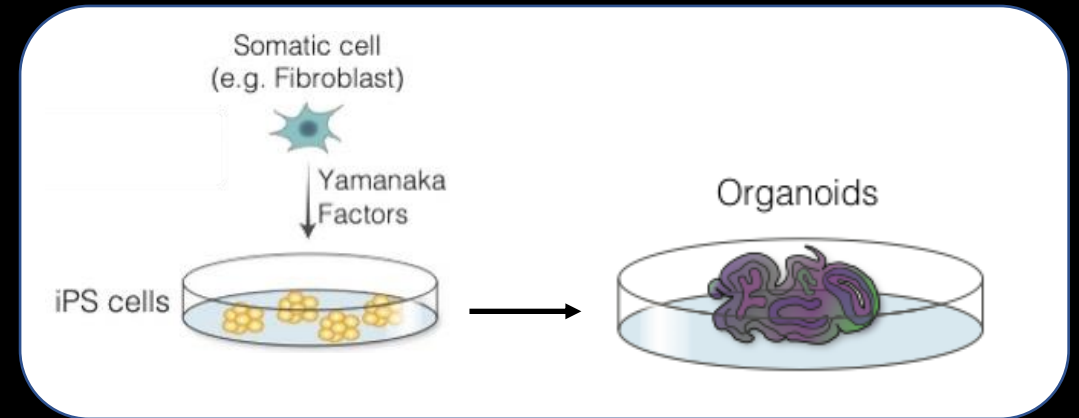
Example data set: scRNA-seq data of brain organoids

LETTER

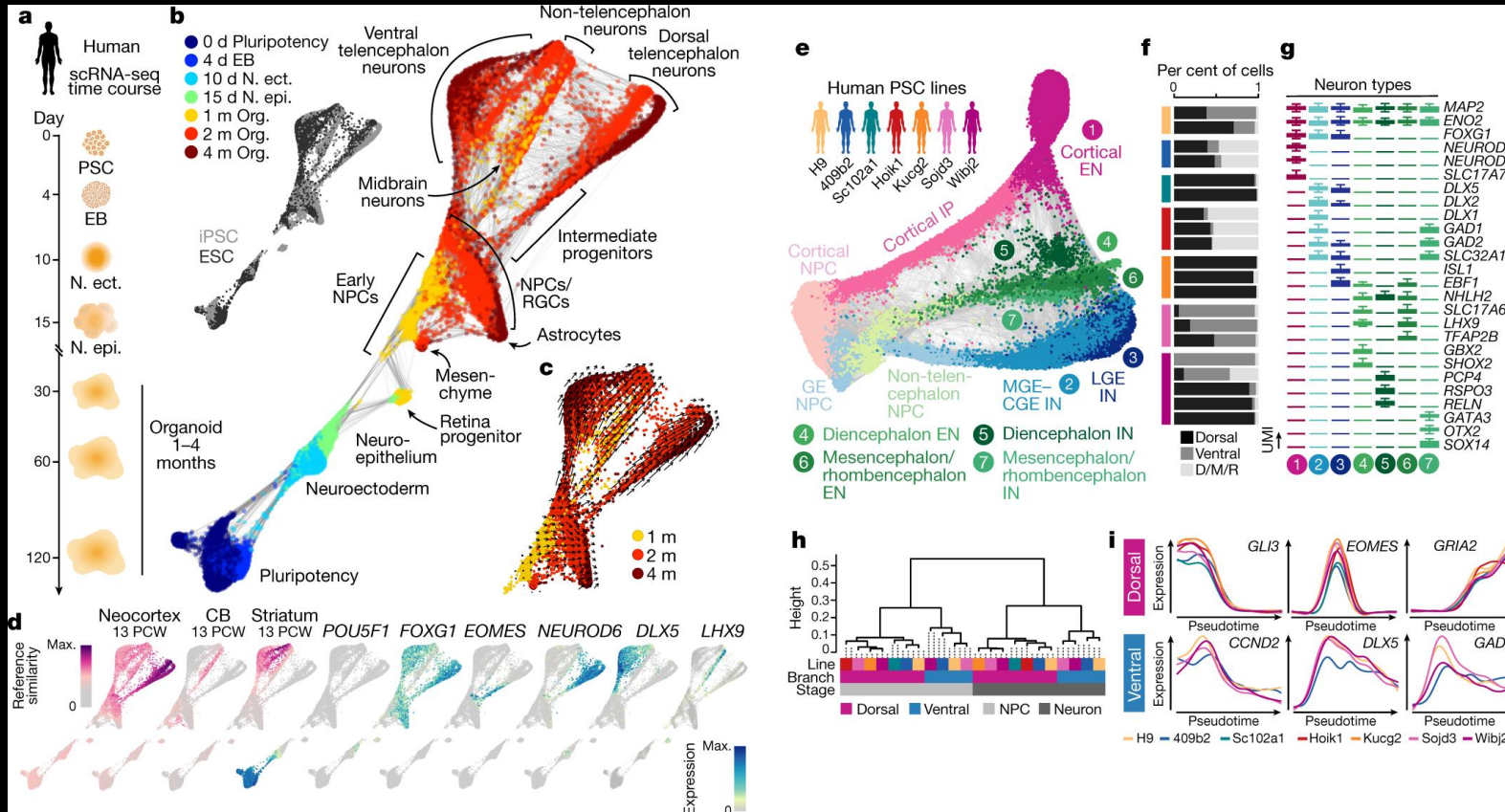
<https://doi.org/10.1038/s41586-019-1654-9>

Organoid single-cell genomic atlas uncovers human-specific features of brain development

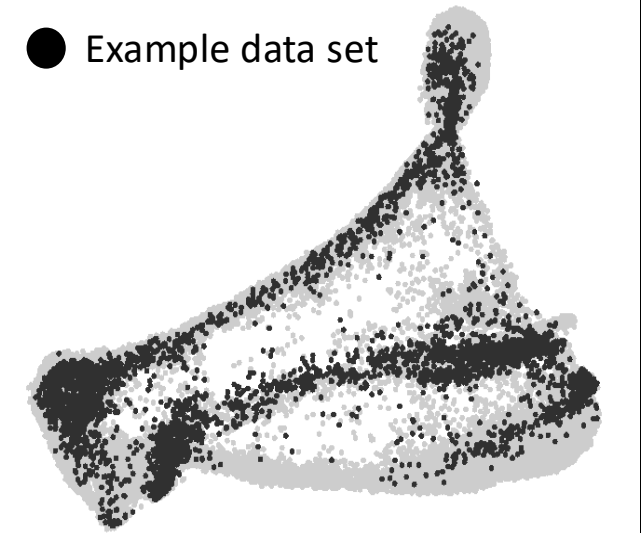
Sabina Kanton^{1,7}, Michael James Boyle^{1,7}, Zhisong He^{1,2,7*}, Malgorzata Santel¹, Anne Weigert¹, Fátima Sanchís-Calleja^{1,2}, Patricia Guijarro³, Leila Sidow¹, Jonas Simon Fleck², Dingding Han³, Zhengzong Qian³, Michael Heide⁴, Wieland B. Huttner⁴, Philipp Khaitovich^{1,3,5}, Svante Pääbo¹, Barbara Treutlein^{1,2*} & J. Gray Camp^{1,6*}



Example data set: scRNA-seq data of brain organoids



● Example data set



Example data set

One 2-month-old organoid

- 4317 cells
- Seurat object
- Preprocessed and annotated
- Also include exonic/intronic count matrices as additional assays

Link to the data (H5AD files for the AnnData objects):

- (1) With total, spliced and unspliced counts: <https://polybox.ethz.ch/index.php/s/GCZzdxkMiTp5ZQH>
- (2) With only total counts but all features: <https://polybox.ethz.ch/index.php/s/GeXSaepAk9fjQqk>

Outlines

- **Introduction**
 - Setup your environment (Python)
 - The example data set
- **Analysis**
 - Brief overview on the data – 5 min
 - Diffusion pseudotime – 10 min
 - Coarse-grained trajectory analysis with PAGA – 10 min
 - RNA velocity analysis with scVelo – 15 min
 - Fate probability estimation with CellRank 2 – 15 min
 - Trajectory analysis with pyURD – 10 min

Standard analysis (with total count matrix)

```
import scanpy as sc
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.pyplot import rc_context

adata_full = sc.read_h5ad('data/DS1_raw.h5ad')
adata_full.layers['counts'] = adata_full.X.copy()
sc.pp.normalize_total(adata_full, target_sum=1e4)
sc.pp.log1p(adata_full)
sc.pp.highly_variable_genes(adata_full, n_top_genes=5000, flavor='seurat_v3', layer='counts')
sc.pp.pca(adata_full, n_comps=30, mask_var="highly_variable")
sc.pp.neighbors(adata_full)
sc.tl.umap(adata_full)
sc.tl.leiden(adata_full, resolution=1, flavor='igraph', n_iterations = 2)

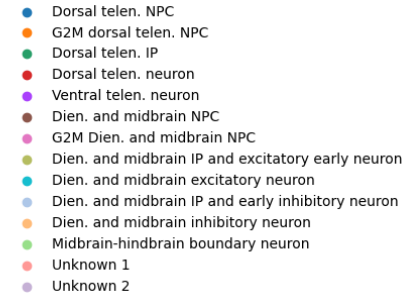
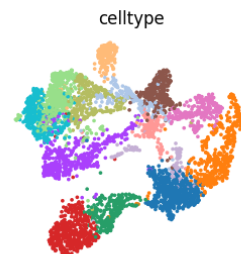
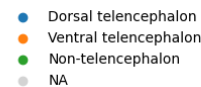
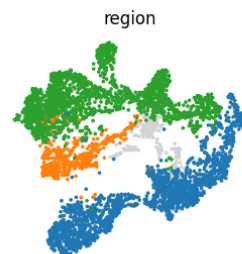
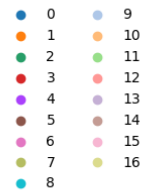
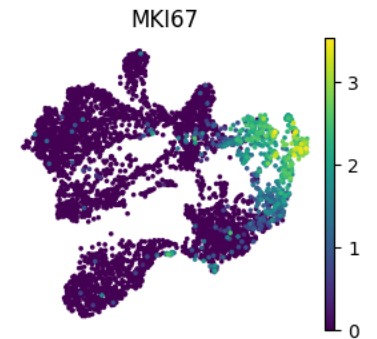
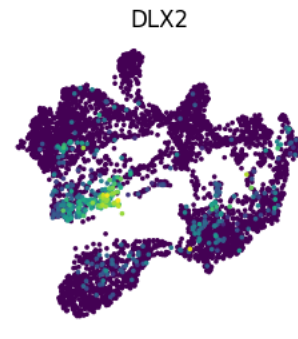
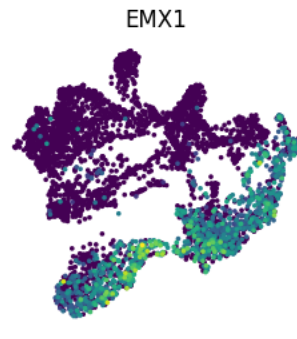
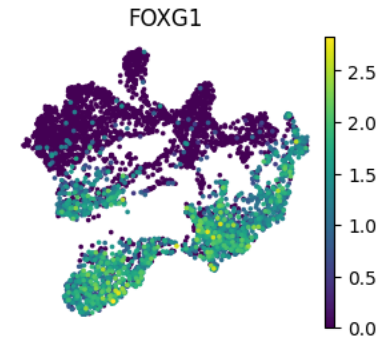
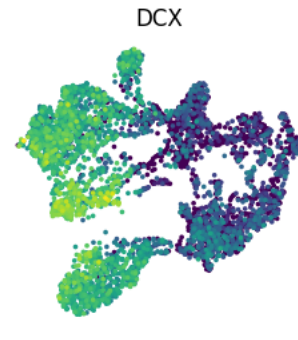
with rc_context({'figure.figsize': (3, 3)}):
    sc.pl.umap(adata_full, color=['SOX2', 'DCX', 'FOXG1', 'EMX1', 'DLX2', 'MKI67'],
               ncols=3, frameon=False, show=True)
    sc.pl.umap(adata_full, color=['leiden', 'region', 'celltype'],
               ncols=3, frameon=False, show=True, wspace = 0.7)
```

Standard analysis (with total count matrix)

```
import scanpy as sc
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.pyplot import figure
```

```
adata_full = sc.read_h5ad('adata_full.h5ad')
adata_full.layers['counts'] = adata_full.X
sc.pp.normalize_total(adata_full)
sc.pp.log1p(adata_full)
sc.pp.highly_variable_genes(adata_full)
sc.pp.pca(adata_full, n_comps=30)
sc.pp.neighbors(adata_full)
sc.tl.umap(adata_full)
sc.tl.leiden(adata_full, resolution=0.8)
```

```
with rc_context({'figure.figsize': (10, 10)}):
    sc.pl.umap(adata_full, color='leiden',
               ncols=3, figsize=(10, 10))
    sc.pl.umap(adata_full, color='region',
               ncols=3, figsize=(10, 10))
    sc.pl.umap(adata_full, color='celltype',
               ncols=3, figsize=(10, 10))
```



Diffusion map and DPT

```
# Load the main object
adata = sc.read_h5ad('data/DS1.h5ad')

# Copy the preprocessed results to the main object
adata.obsm = adata_full[adata.obs_names, :].obsm.copy()
adata.obsp = adata_full[adata.obs_names, :].obsp.copy()
adata.uns = adata_full[adata.obs_names, :].uns.copy()

# Run Diffusion Map
sc.pp.neighbors(adata, n_neighbors=50, n_pcs=30, use_rep='X_pca')
sc.tl.diffmap(adata, n_comps=20)
```

```
# Infer the root
idx_subset = np.where(
    np.isin(
        adata.obs['celltype'],
        [
            'Dorsal telen. NPC',
            'G2M dorsal telen. NPC',
            'Dien. and midbrain NPC',
            'G2M Dien. and midbrain NPC'
        ]
    ))[0]
idxs_rand_root = np.apply_along_axis(
    lambda x: random_root(adata, idx_subset=idx_subset),
    1, np.array(range(1000))[:, None]
)
adata.uns['iroot'] = np.argmax(np.bincount(idxs_rand_root))
```

```
# Method to infer the root
import random
def random_root(
    adata, seed=None, neighbors_key=None, idx_subset=None
):
    if seed is not None:
        random.seed(seed)
    iroot_bak = None
    if 'iroot' in adata.uns.keys():
        iroot_bak = adata.uns['iroot'].copy()
    dpt_bak = None
    if 'dpt_pseudotime' in adata.obs.columns:
        dpt_bak = adata.obs['dpt_pseudotime'].copy()

    idx = np.random.choice(list(range(adata.shape[0])))
    adata.uns['iroot'] = idx
    sc.tl.dpt(adata, neighbors_key=neighbors_key)
    dpt = adata.obs['dpt_pseudotime']
    if idx_subset is not None:
        dpt = dpt.iloc[idx_subset]
    idx_max_dpt = np.argmax(dpt)
    if idx_subset is not None:
        idx_max_dpt = idx_subset[idx_max_dpt]

    del adata.uns['iroot']
    del adata.obs['dpt_pseudotime']
    if iroot_bak is not None:
        adata.uns['iroot'] = iroot_bak.copy()
    if dpt_bak is not None:
        adata.obs['dpt_pseudotime'] = dpt_bak.copy()

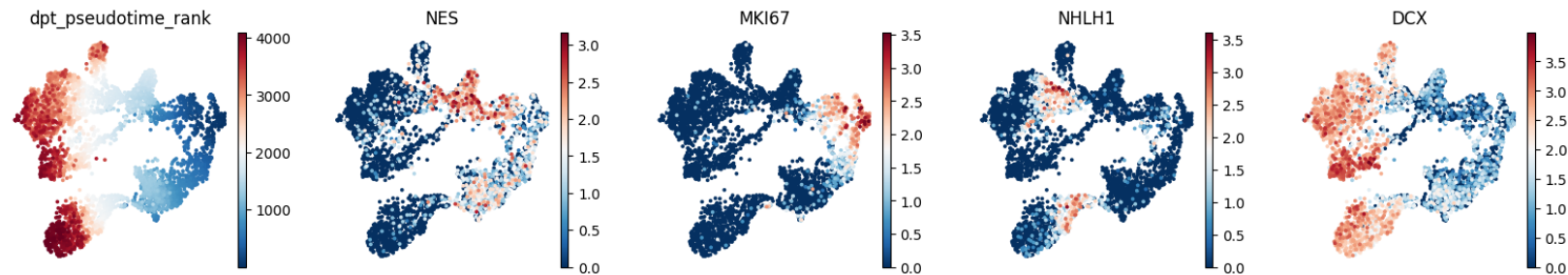
    return idx_max_dpt
```

Diffusion map and DPT

```
# Run DPT
sc.tl.dpt(adata, n_dcs=20)

# Rank DPT
from scipy.stats import rankdata
adata.obs['dpt_pseudotime_rank'] = rankdata(adata.obs['dpt_pseudotime'])

# Visualization of DPT on UMAP
with rc_context({'figure.figsize': (3, 3)}):
    sc.pl.umap(
        adata,
        color=['dpt_pseudotime_rank', 'NES', 'MKI67', 'NHLH1', 'DCX'],
        color_map='RdBu_r',
        ncols=5, frameon=False
    )
```



Coarse-grained trajectory analysis with PAGA

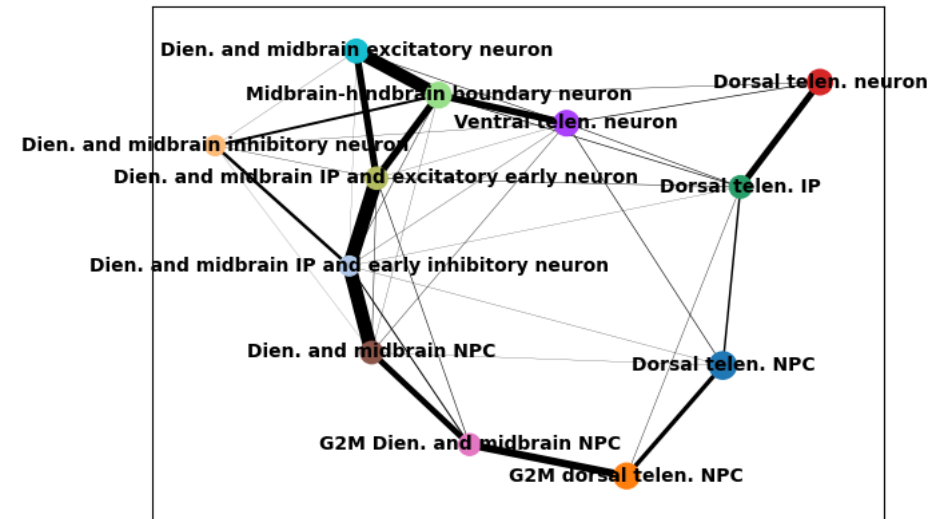
```
sc.pp.neighbors(adata, n_neighbors=50, n_pcs=20, use_rep='X_diffmap')
adata.obs['celltype'] = adata.obs['celltype'].cat.remove_unused_categories()
sc.tl.paga(adata, groups='celltype')
sc.pl.paga(adata)
```

Perform PAGA and visualize estimated cell type connectivities

```
from scipy import sparse

connected = adata.uns['paga']['connectivities'] > 0.1
connected = (connected + connected.T) > 0
idx_row, idx_col, dat = sparse.find(connected)
idx = (idx_row >= idx_col)
connected_celltypes = pd.DataFrame(
    {
        'node1': adata.obs['celltype'].cat.categories[idx_row[idx]],
        'node2': adata.obs['celltype'].cat.categories[idx_col[idx]]
    }
)
connected_celltypes
```

Check connected cell types



RNA velocity analysis with scVelo

```
import scvelo as scv

# Filter features
adata.raw = adata
scv.pp.filter_and_normalize(
    adata, min_shared_counts=20,
    min_shared_cells=20
)

# Estimate velocity
sc.pp.neighbors(
    adata, use_rep='X_pca', n_neighbors=50
)
scv.pp.moments(adata, n_neighbors=50)
scv.tl.velocity(adata, mode='stochastic')
scv.tl.velocity_graph(adata)

# Visualization
fig, ax = plt.subplots(1, 3, figsize=(12, 4))
scv.pl.velocity_embedding_stream(
    adata, ax=ax[0], basis='umap', color='celltype', frameon=False, show=False
)
scv.pl.velocity_embedding_grid(
    adata, ax=ax[1], basis='umap', color='celltype', frameon=False, arrow_size=2, arrow_length=2, show=False
)
scv.pl.velocity_embedding(
    adata, ax=ax[2], basis='umap', color='celltype', frameon=False, arrow_size=2, arrow_length=2, show=False
)
plt.show()
```

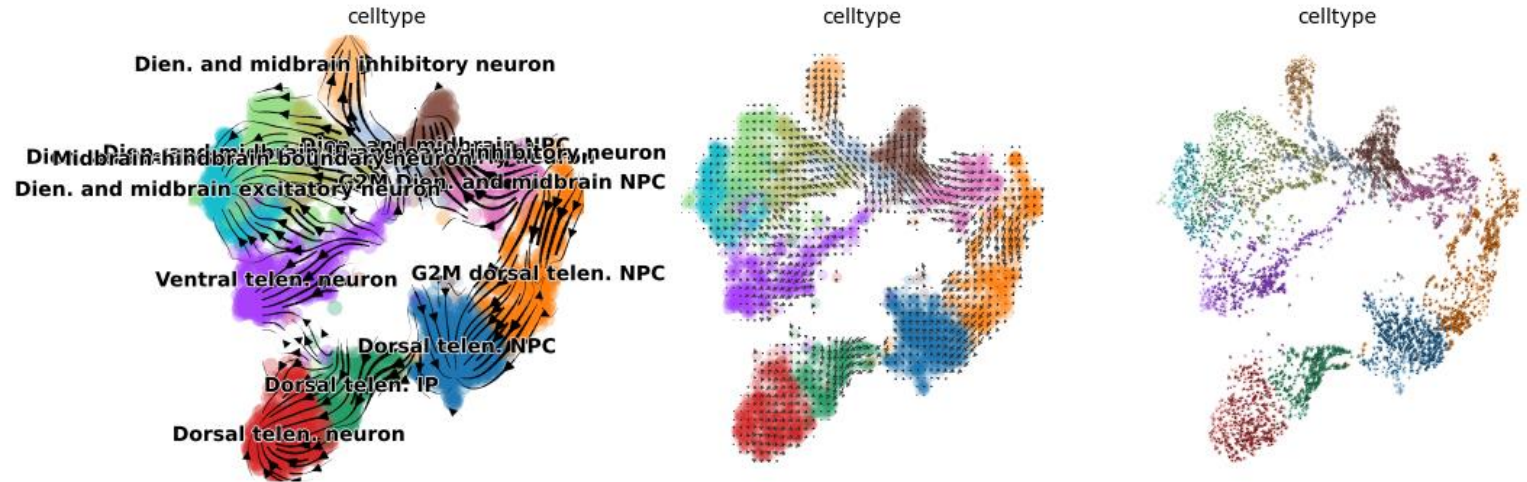
RNA velocity analysis with scVelo

```
import scvelo as scv

# Filter features
adata.raw = adata
scv.pp.filter_and_normalize(
    adata, min_shared_counts=20,
    min_shared_cells=20
)

# Estimate velocity
sc.pp.neighbors(
    adata, use_rep='X_pca', n_neighbors=50
)
scv.pp.moments(adata, n_neighbors=50)
scv.tl.velocity(adata, mode='stochastic')
scv.tl.velocity_graph(adata)

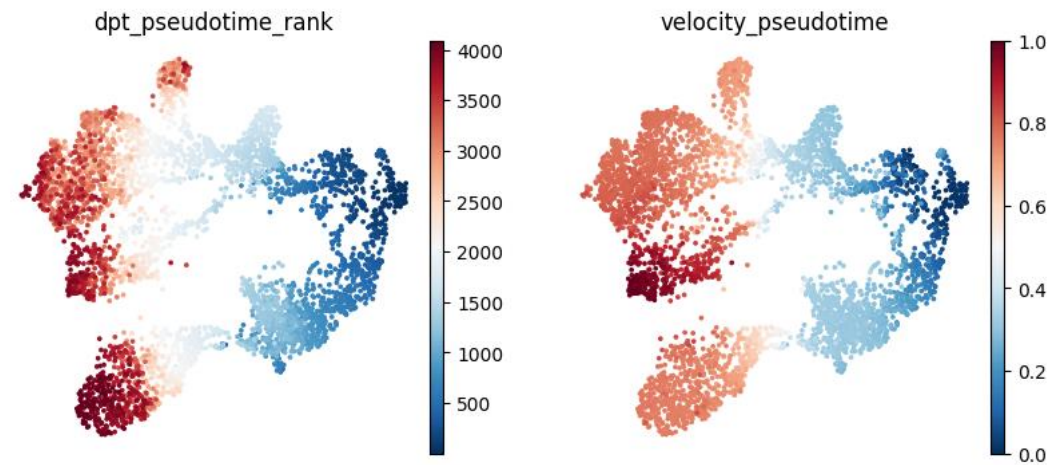
# Visualization
fig, ax = plt.subplots(1, 3, figsize=(12, 4))
scv.pl.velocity_embedding_stream(
    adata, ax=ax[0], basis='umap', color='celltype', frameon=False, show=False
)
scv.pl.velocity_embedding_grid(
    adata, ax=ax[1], basis='umap', color='celltype', frameon=False, arrow_size=2, arrow_length=2, show=False
)
scv.pl.velocity_embedding(
    adata, ax=ax[2], basis='umap', color='celltype', frameon=False, arrow_size=2, arrow_length=2, show=False
)
plt.show()
```



RNA velocity analysis with scVelo

```
# Estimate velocity pseudotime
scv.tl.velocity_pseudotime(adata)

# Visualize both the DPT and velocity pseudotime
with rc_context({'figure.figsize': (4, 4)}):
    sc.pl.umap(adata, color=['dpt_pseudotime_rank', 'velocity_pseudotime'],
               cmap='RdBu_r', frameon=False, ncols=2)
```

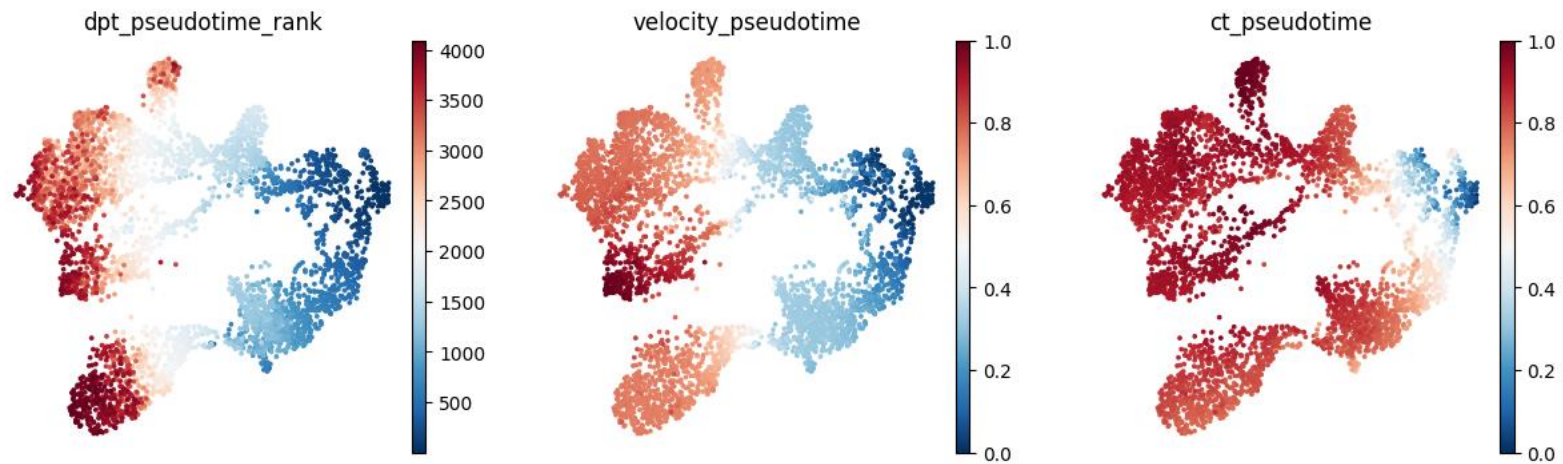


CytoTRACE with its implementation in CellRank2

```
from cellrank.kernels import CytoTRACEKernel

ctk = CytoTRACEKernel(adata).compute_cytotrace()

with rc_context({'figure.figsize': (4, 4)}):
    sc.pl.umap(adata, color=['dpt_pseudotime_rank', 'velocity_pseudotime', 'ct_pseudotime'],
               cmap='RdBu_r', frameon=False, ncols=3)
```



Fate probability estimation with CellRank2

```
import cellrank as cr

pk = cr.kernels.PseudotimeKernel(adata, time_key='dpt_pseudotime').compute_transition_matrix()
ck = cr.kernels.ConnectivityKernel(adata).compute_transition_matrix()
vk = cr.kernels.VelocityKernel(adata).compute_transition_matrix()
ctk = cr.kernels.CytoTRACEKernel(adata).compute_cytotrace().compute_transition_matrix()
combined_kernel = 0.4 * vk + 0.3 * pk + 0.2 * ck + 0.1 * ctk
```

Generate hybrid kernels summarizing transcriptomic similarity, pseudotime, velocity and CytoTRACE

```
g = cr.estimators.GPCCA(combined_kernel)
g.fit(n_states=15, cluster_key='celltype')
g.predict_terminal_states(method='top_n', n_states=10)
g.plot_macrostates(which='terminal')
## Google Colab users may fail at this step. It won't affect the following steps.
```

Predict terminal states

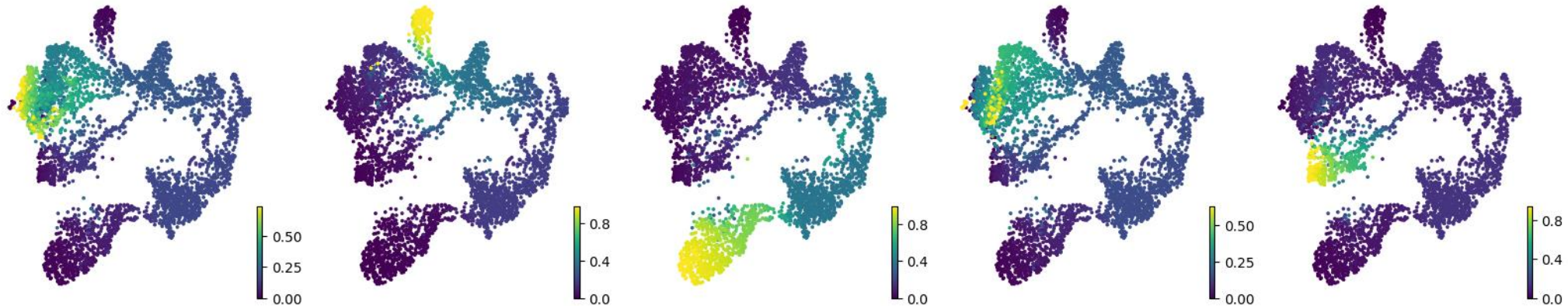
```
g = cr.estimators.GPCCA(combined_kernel)
neuron_types = [x for x in adata.obs['celltype'].cat.categories if x.endswith('neuron') and 'early' not in x]
terminal_states = [adata[adata.obs['celltype'] == x,
:].obs['dpt_pseudotime'].sort_values(ascending=False)[:30].index
for x in neuron_types]
terminal_states = dict(zip(neuron_types, terminal_states))
g.set_terminal_states(terminal_states)
g.plot_macrostates(which='terminal')
```

Manually assign terminal states (each neuron type with the highest diffusion pseudotime)

Fate probability estimation with CellRank2

```
g.compute_fate_probabilities()
with rc_context({'figure.figsize': (4, 4)}):
    g.plot_fate_probabilities(legend_loc='right', basis='X_umap', same_plot=False)
```

fate probabilities Diencephalic and midbrain excitatory neuron fate probabilities Diencephalic and midbrain inhibitory neuron fate probabilities Dorsal telencephalic neuron fate probabilities Midbrain-hindbrain boundary neuron fate probabilities Ventral telencephalic neuron



```
# Save the results
prob = pd.DataFrame(g.fate_probabilities).set_index(g.terminal_states.index).set_axis(g.terminal_states.cat.categories, axis=1)
adata.obs = pd.concat([adata.obs, prob.set_axis(['CR_' + x for x in prob.columns], axis=1)], axis=1)
for obsm in adata.obsm.keys():
    if type(adata.obsm[obsm]).__name__ in ['Lineage', 'LineageView']:
        adata.uns[str(obsm)+'_lineage'] = {
            'colors': adata.obsm[obsm].colors,
            'names': adata.obsm[obsm].names
        }
        adata.obsm[obsm] = np.array(adata.obsm[obsm])
adata.write_h5ad('data/DS1_processed.h5ad')
```

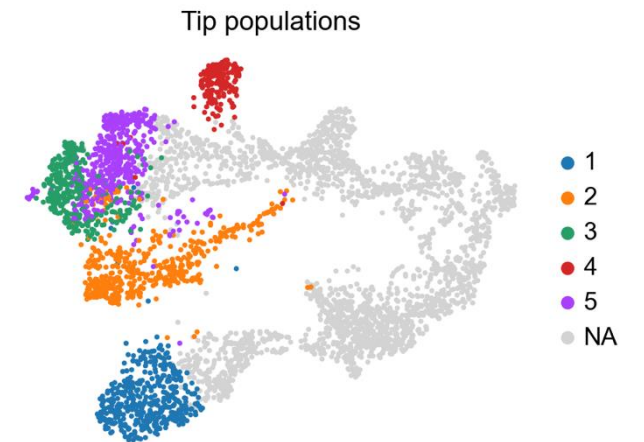
(Optional) Trajectory analysis with pyURD

```
# Load package and data
import pyurd
adata_urd = sc.read_h5ad("data/DS1_processed.h5ad")
```

```
# Root cells: all NPC populations
ROOT_TYPES = [
    "Dorsal telen. NPC",
    "G2M dorsal telen. NPC",
    "Dien. and midbrain NPC",
    "G2M Dien. and midbrain NPC",
]
root_cells = adata_urd.obs_names[adata_urd.obs["celltype"].isin(ROOT_TYPES)].tolist()

# Tip populations: five terminal neuronal lineages
TIP_MAP = {
    "1": "Dorsal telen. neuron",
    "2": "Ventral telen. neuron",
    "3": "Dien. and midbrain excitatory neuron",
    "4": "Dien. and midbrain inhibitory neuron",
    "5": "Midbrain-hindbrain boundary neuron",
}
adata_urd.obs["tip"] = np.nan
for tip_id, ct in TIP_MAP.items():
    mask = adata_urd.obs["celltype"] == ct
    adata_urd.obs.loc[mask, "tip"] = tip_id

# Visualize the tip populations
sc.pl.umap(adata_urd, color="tip", title="Tip populations")
```



(Optional) Trajectory analysis with pyURD

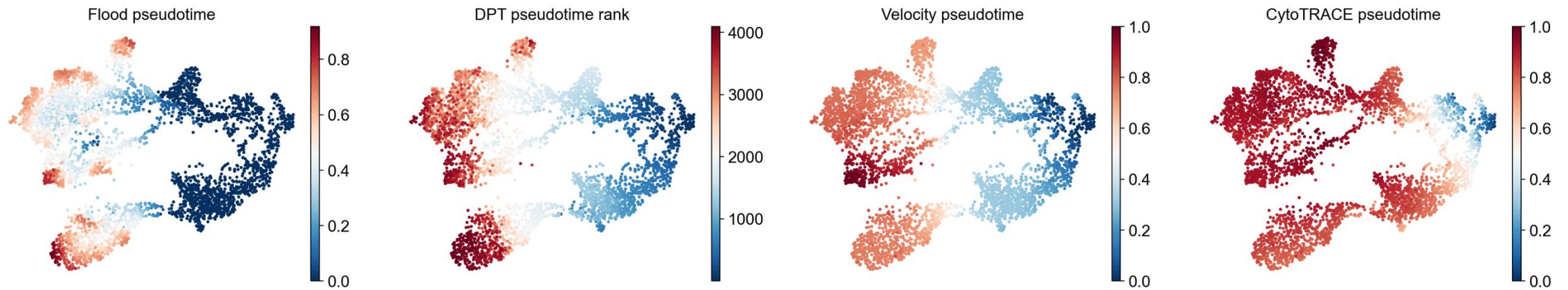
```
# Diffusion map based transition matrix denoising
transition_key = pyurd.prepare_connectivities(adata_urd, n_neighbors=50, n_dcs=30)

# Flood simulations
floods = pyurd.flood_pseudotime(
    adata_urd,
    root_cells=root_cells,
    n=50, minimum_cells_flooded=2, transition_key=transition_key, verbose=False, n_jobs=-1,
)

# Summarize flood simulations into pseudotime
pyurd.flood_pseudotime_process(
    adata_urd,
    floods=floods,
    floods_name="pseudotime", max_frac_na=0.4, stability_div=10,
)
```

(Optional) Trajectory analysis with pyURD

```
# Visualize and compare different pseudotimes
sc.pl.umap(
    adata_urd,
    color=["pseudotime", 'dpt_pseudotime_rank', 'velocity_pseudotime', 'ct_pseudotime'],
    color_map="RdBu_r", ncols=3, frameon=False,
    title=["Flood pseudotime", "DPT pseudotime rank", "Velocity pseudotime", "ytoTRACE pseudotime"]
)
corr_df = adata_urd.obs[["pseudotime", "dpt_pseudotime_rank", "velocity_pseudotime", "ct_pseudotime"]].corr(method='spearman')
print(corr_df.to_string())
```



(Optional) Trajectory analysis with pyURD

```
# Fit logistic parameters:
logistic_params = pyurd.pseudotime_determine_logistic(
    adata_urd, pseudotime_key="pseudotime", optimal_cells_forward=20, max_cells_back=40, do_plot=True, verbose=True,
)

# Build the biased transition matrix
biased_tm = pyurd.pseudotime_weight_transition_matrix(
    adata_urd, pseudotime_key="pseudotime", logistic_params=logistic_params, transition_key=transition_key, chunk_size=500,
)
```

```
# Register tip cells
pyurd.load_tip_cells(adata_urd, tips_key="tip")

# Simulate 10 000 random walks per tip
walks = pyurd.simulate_random_walks_from_tips(
    adata_urd, tip_group_key="tip", root_cells=root_cells, transition_matrix=biased_tm, n_per_tip=10_000, root_visits=3,
    max_steps=5000, verbose=True,
)

# Convert walks to per-cell visitation frequencies stored in adata_urd.obs
pyurd.process_random_walks_from_tips(adata_urd, walks_dict=walks, verbose=True)
```

(Optional) Trajectory analysis with pyURD

```
# Join the five tip trajectories by finding the pseudotime at which their visitation patterns diverge
pyurd.build_tree(
    adata_urd, pseudotime_key="pseudotime", tips_use=list(TIP_MAP.keys()), # ["1", "2", "3", "4", "5"]
    divergence_method="preference", # uses Hartigan dip test
    weighted_fusion=True, cells_per_pseudotime_bin=80, bins_per_pseudotime_window=5, minimum_visits=10,
    visit_threshold=0.7, p_thresh=0.01, dendro_node_size=100, verbose=True,
)

# Assign human-readable names to terminal segments
pyurd.name_segments(
    adata_urd, segments=["1", "2", "3", "4", "5"],
    segment_names=[
        "dTelen Neuron",
        "vTelen Neuron",
        "Dien/MB Exc. Neuron",
        "Dien/MB Inh. Neuron",
        "MHB Neuron",
    ], short_names=["dTEN", "vTEN", "DiMB-E", "DiMB-I", "MHB"],
)

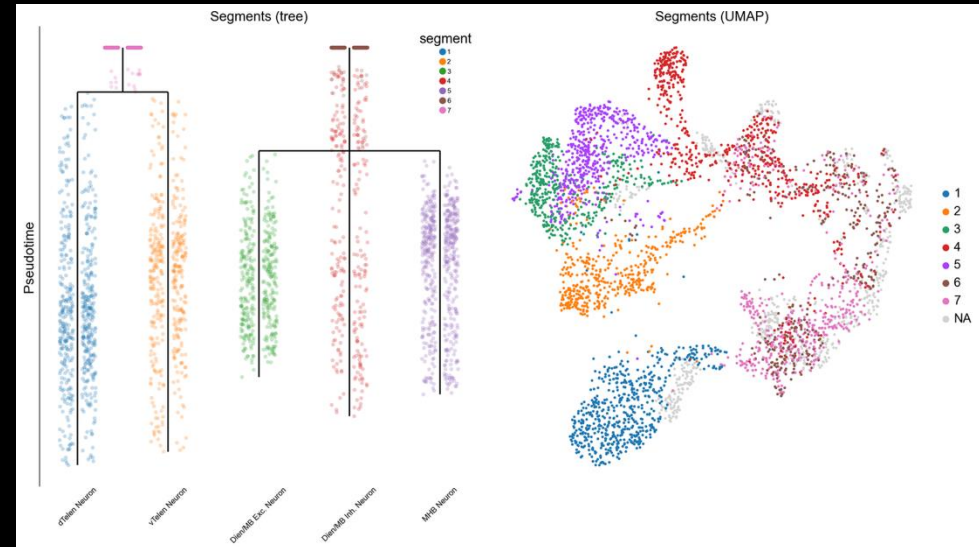
# Visualize the tree and segments on the UMAP
fig, axes = plt.subplots(1, 2, figsize=(14, 8))
pyurd.plot_tree(adata_urd, label="segment", title="Segments (tree)", ax=axes[0], legend=True, cell_size=4)
sc.pl.umap(adata_urd, color="segment", title="Segments (UMAP)", ax=axes[1], show=False)
axes[1].set_aspect("auto")
plt.tight_layout()
plt.show()
```

(Optional) Trajectory analysis with pyURD

```
# Join the five tip trajectories by finding the pseudotime at which their visitation patterns diverge
pyurd.build_tree(
    adata_urd, pseudotime_key="pseudotime", tips_use=list(TIP_MAP.keys()), # ["1", "2", "3", "4", "5"]
    divergence_method="preference", # uses Hartigan dip test
    weighted_fusion=True, cells_per_pseudotime_bin=80, bins_per_pseudotime_window=5, minimum_visits=10,
    visit_threshold=0.7, p_thresh=0.01, dendro_node_size=100, verbose=True,
)
```

```
# Assign human-readable names to terminal segments
pyurd.name_segments(
    adata_urd, segments=["1", "2", "3", "4", "5"],
    segment_names=[
        "dTelen Neuron",
        "vTelen Neuron",
        "Dien/MB Exc. Neuron",
        "Dien/MB Inh. Neuron",
        "MHB Neuron",
    ], short_names=["dTN", "vTN", "DiMB-E", "DiMB-I", "MHB"],
)
```

```
# Visualize the tree and segments on the UMAP
fig, axes = plt.subplots(1, 2, figsize=(14, 8))
pyurd.plot_tree(adata_urd, label="segment", title="Segments (tree)", ax=axes[0], legend=True, cell_size=4)
sc.pl.umap(adata_urd, color="segment", title="Segments (UMAP)", ax=axes[1], show=False)
axes[1].set_aspect("auto")
plt.tight_layout()
plt.show()
```



Questions?